Constrains in SQL

What is a Constraint?

A constraint is a rule that you define on a table that restricts the values in that table.

- They can be added to a table or a view when you create it, or after it's created. You do this by specifying a few keywords and some information about the columns and rules you want to set.
- They improve the quality and integrity of the data. We can help ensure that the data meets the rules that we have set for it (e.g. an employee's salary is not null).
- So, constraints are created to protect the integrity of the data and improve the quality of the data.
- A CONSTRAINT clause is an optional part of a CREATE TABLE statement or ALTER TABLE statement. A constraint is a rule to which data must conform. Constraint names are optional.

A CONSTRAINT can be one of the following:

- **a column-level constraint (lnline Constraint)**

  Column-level constraints refer to a single column in the table and do not specify a column name (except check constraints). They refer to the column that they follow.

- **a table-level constraint (out of line)**

  Table-level constraints refer to one or more columns in the table. Table-level constraints specify the names of the columns to which they apply. Table-level CHECK constraints can refer to 0 or more columns in the table.

What Are the Types of Constraints in SQL?

There are five different types of SQL constraints.

They are:

- **Primary Key Constraint**: this ensures all rows have a unique value and cannot be NULL, often used as an identifier of a table's row.
- **Foreign Key Constraint**: this ensures that values in a column (or several columns) match values in another table's column/s.
- **Unique Constraint**: this ensures all rows have a unique value.
- **Not Null Constraint**: this ensures a value cannot be NULL.
- **Check Constraint**: this ensures a value meets a specific condition.

How Can You Create a Constraint?

There are two places you can create a constraint:

- When the table is created, as part of the CREATE TABLE (or CREATE VIEW) statement
- After the table is created, as part of the ALTER TABLE (or ALTER VIEW) statement

In addition to this, there are two ways you can define a constraint when adding it as part of the CREATE statement:

- **Inline: next to the name of the column(Column level)**
- **Out of line: at the end of the CREATE statement (Table –level)**

As part of the constraint type definitions below, I'll show you how to create a constraint using both methods for a CREATE statement (inline and out of line), and using an ALTER statement.

For a CREATE statement, which way is better – inline or out of line?

I think the **out of line** method is better, because you can do some helpful things such as add multiple columns, and define a name. But this will become clearer when we see the examples!

How Should You Name Your Constraints?

Your constraint names should be able to be used to identify which table and column they relate to, as well as the type.

This makes it easier to know by looking at the name what type it is, and what it refers to.

I recommend that a constraint name has a combination of:

- A two-letter term to indicate the type of constraint
- An abbreviated name of the table
- An abbreviate name of the column(s) or rule for the constraint

Each of these would be separated by an underscore.

For example, the constraint name "pk_emp_id" is a primary key (pk), refers to the employee table (emp), and refers to the ID column (id).

1. Primary Key Constraint

A primary key constraint, or primary key, is often referred to as the identifier of a row.

Creating a primary key on a column or set of columns means that each row must have a unique value for this column(s), and it cannot be NULL.

A primary key is probably the most common constraint.

It allows you to have an identifier of a row, which is unique.

There is often, but not always, a single column that is the "primary key", which means it has a primary key constraint applied to it. It is often a kind of number field, such as "id" or "_no".

A table can only have one primary key on it. If you try to create a second primary key, you'll get an error.

Also, a primary key works like a combination of a Unique Constraint and a Not Null Constraint. So if you need to add additional unique rules to your table, you can use a Unique Constraint.

Creating a Primary Key

The syntax looks like this:

```
CONSTRAINT constraint_name PRIMARY KEY constraint_parameters
```

This is the same whether it's defined inline or out of line.

Another way to define a primary key is to add the words PRIMARY KEY after the column name when defining it inline.

Primary Key Restrictions

There are some restrictions that apply to primary keys:

- No value in a column that is in a primary key can exist more than once in the column (aka no duplicate values).
- No values in primary key columns can be NULL.
- A table or view can have only one primary key.
- The columns in the primary key cannot be any of the following types: LOB, BFILE, TIMESTAMP WITH TIME ZONE, LONG, LONG RAW, VARRAY, NESTED TABLE, REF, or a user defined type.
- A composite primary key can't have more than 32 columns.
- The same column or combination of columns cannot be part of a primary key and a unique constraint.

Example 1 – Inline Primary Key

The data types used here are Oracle-specific, so change them to fit your database.

This example shows you how to define a primary key on a table as it's created, using the inline method.

```
CREATE TABLE employee (
  employee_id NUMBER(10) PRIMARY KEY,
  first_name VARCHAR2(20),
  last_name VARCHAR2(20),
  salary NUMBER(10),
```

```
  hire_date DATE
);
```

This uses the simple syntax of just the keywords PRIMARY KEY. It will create a primary key on the employee_id column.

Example 2 – Inline Primary Key with a Name

In this example, we'll look at how you can create a primary key on a table using the inline method and providing a name.

We can use the previous example table (assuming the table does not exist):

```
CREATE TABLE employee (
  employee_id NUMBER(10) CONSTRAINT pk_emp_id PRIMARY KEY,
  first_name VARCHAR2(20),
  last_name VARCHAR2(20),
  salary NUMBER(10),
  hire_date DATE
);
```

The table is created, and a primary key constraint with the name pk_emp_id is created on the table.

I prefer this method compared to the first example, as there is a more readable and logical name for the primary key.

Example 3 – Out of Line

Another way to create a constraint on a table is to use the out of line method.

```
CREATE TABLE employee (
  employee_id NUMBER(10),
  first_name VARCHAR2(200),
  last_name VARCHAR2(200),
  salary NUMBER(10),
  hire_date DATE,
  CONSTRAINT pk_emp_id PRIMARY KEY (employee_id)
);
```

This method also allows you to name the primary key, which is named pk_emp_id. We specify the column name in brackets, so we know which column it refers to.

Example 4 – Alter Table

You can add a primary key constraint to an existing table using the ALTER TABLE statement.

```
ALTER TABLE employee
ADD CONSTRAINT pk_emp_id PRIMARY KEY(employee_id);
```

After the ALTER TABLE ADD part, the syntax is the same as an out of line constraint on a table. Here, we've added a primary key constraint onto the employee_id column.

Example 5 – Composite Primary Key

If you want to use more than one column in your primary key, then this needs to be defined as an out of line constraint. This is because you can't reference more than one column in an inline constraint.

An example of this would be:

```
CREATE TABLE employee (
  employee_id NUMBER(10),
  first_name VARCHAR2(20),
  last_name VARCHAR2(20),
  salary NUMBER(10),
  hire_date DATE,
  CONSTRAINT pk_emp_flname PRIMARY KEY (first_name, last_name)
);
```

Example 6 – Inserting Data

Now, let's try and insert some data into our table that has a primary key on the employee_id column.

```
INSERT INTO employee
(employee_id, first_name, last_name, salary, hire_date)
VALUES (1, 'John', 'Smith', 50000, TO_DATE('20-OCT-2020', 'DD-MON-YYYY'));

INSERT INTO employee
(employee_id, first_name, last_name, salary, hire_date)
VALUES (2, 'Sarah', 'Jones', 65000, TO_DATE('04-DEC-2020', 'DD-MON-YYYY'));
```

Now let's insert a row with the same ID as an existing record. This should not be allowed, as it violates our primary key constraint.

```
INSERT INTO employee
(employee_id, first_name, last_name, salary, hire_date)
VALUES (2, 'Adam', 'Brown', 42000, TO_DATE('13-JAN-2020', 'DD-MON-YYYY'));
```

```
ORA-00001: unique constraint (ben.pk_emp_flname) violated
```

NOTE:

This has failed because a record with an employee_id value of 2 already exists. We will have to change the value in our statement to another value for it to be inserted correctly.

The ORA-00001 is an Oracle error, and we'll get a similar "unique constraint violated" error in other databases.

2. Foreign Key Constraint

A foreign key constraint is used to create a link between a column(s) in one table and a primary key or unique key in another table.

It allows you to enforce referential integrity, which means that a record in one table relates to a record in another table.

It's an optional constraint, like all constraints, but it's one I've seen implemented a lot. If you want to ensure you have quality data in your database, your tables should have foreign key constraints on them.

You can actually run SELECT queries that use joins without having foreign keys. But having foreign keys will ensure you relate your data correctly, which is one of the main benefits of having a relational database.

Parent and Child Records

Often when we refer to records with foreign keys, we say they are "child" records. The "parent" record is the record that has the column the foreign key refers to.

For example, if an employee related to a department, the department would be the parent record, and the employee would be the child record.

Creating a Foreign Key

Creating a foreign key is very similar to creating a primary key. But we need to add some extra information to mention the table and column it refers to.

The syntax looks like this:

```
CONSTRAINT constraint_name REFERENCES constraint_parameters
```

This is the same whether it's defined inline or out of line.

Foreign Key Restrictions

There are a few restrictions when creating a foreign key:

- The columns in the foreign key cannot be any of the following types: LOB, BFILE, TIMESTAMP WITH TIME ZONE, LONG, LONG RAW, VARRAY, NESTED TABLE, REF, or a user defined type.
- The primary key or unique key referenced by the foreign key must already be created on the referenced table.
- A composite foreign key can't have more than 32 columns.
- Both tables (the table that has the primary key and the table that has the foreign key) must be on the same database.
- You can't define a foreign key when creating a table using CREATE TABLE AS with a subquery in the AS clause. To do this, you'll need to create the table first, then use the ALTER TABLE statement to add one.

ON DELETE

The ON DELETE clause is a clause of a foreign key. It lets you determine how you want to treat referenced data when you delete the parent record.

There are two options:

- **ON DELETE SET NULL:** When you delete the parent record, then all child records will have the referenced column set to NULL.
- **ON DELETE CASCADE:** When you delete the parent record, then all child records will be deleted as well.

By default (if you don't specify the ON DELETE clause), the database will not let you delete parent records if a child record exists.

I'll show you some examples of this shortly.

The data types used here are Oracle-specific, so change them to fit your database.

Example 1 – Inline Foreign Key

One way to create a foreign key is to declare it inline, or next to the column itself.

```
CREATE TABLE employee (
  employee_id NUMBER(10),
  first_name VARCHAR2(20),
  last_name VARCHAR2(20),
  salary NUMBER(10),
  hire_date DATE,
  department_id NUMBER(10) CONSTRAINT fk_emp_deptid
  REFERENCES department(dept_id)
);
```

This example has the word CONSTRAINT after the column data type definition. We then name the constraint “fk_emp_deptid”, use the REFERENCES keyword, and then specify the table and column in that table that this column refers to.

The thing to remember here is that the column name inside the brackets is the column name from the department table, not this table. So there would be a column in the department table called dept_id.

Example 2 – Out of Line

This example shows you how to declare a foreign key constraint using the out of line method.

```
CREATE TABLE employee (
  employee_id NUMBER(10),
  first_name VARCHAR2(200),
  last_name VARCHAR2(200),
  salary NUMBER(10),
  hire_date DATE,
  department_id NUMBER(10),
  CONSTRAINT fk_emp_deptid
  FOREIGN KEY (department_id)
  REFERENCES department(dept_id)
);
```

The difference with this one is that we have not declared the constraint on the same line as the column.

We need to specify FOREIGN KEY, and then the column name we’re referring to.

This method also lets us create a foreign key on two columns, if we wanted to.

Example 3 – Delete Record

This example will show you what happens when we delete a parent record (the one with the primary key) when we have a related foreign key record.

Our table was created like this:

```
CREATE TABLE employee (
  employee_id NUMBER(10),
  first_name VARCHAR2(20),
  last_name VARCHAR2(20),
  salary NUMBER(10),
  hire_date DATE,
  department_id NUMBER(10),
  CONSTRAINT fk_emp_deptid
  FOREIGN KEY (department_id)
  REFERENCES department(dept_id)
);
```

We also have a department table. The data in both tables looks like this:

Department

dept_id	department_name
1	Sales
2	HR
3	Finance

Employee

employee_id	first_name	department_id	(other columns…)
1	John	2	…
2	Sarah	3	…
3	Adam	1	…
4	Debbie	1	…

Now, let's try to delete department ID 3.

```
DELETE FROM department
WHERE dept_id = 3;
ORA-02292: integrity constraint (ben.fk_emp_deptid) violated - child record found
```

We get this error because there are employee records that have a department ID record. The ORA-02292 is an Oracle error, and we'll get a similar "integrity constraint violated" error in other databases.

Example 4 – On Delete Set Null

This example will show you how to use the "on delete set null" parameter.

We would create our employee table like this:

```
CREATE TABLE employee (
  employee_id NUMBER(10),
  first_name VARCHAR2(20),
  last_name VARCHAR2(20),
  salary NUMBER(10),
  hire_date DATE,
  department_id NUMBER(10),
  CONSTRAINT fk_emp_deptid
  FOREIGN KEY (department_id)
  REFERENCES department(dept_id)
  ON DELETE SET NULL
);
```

It doesn't matter if the constraint is inline or out of line.

Now, let's say we have sample data in the department and employee table that looks like this:

Department

dept_id	department_name
1	Sales
2	HR
3	Finance

Employee

employee_id	first_name	department_id	(other columns…)
1	John	2	…
2	Sarah	3	…
3	Adam	1	…
4	Debbie	1	…

If we delete department 3 from our department table, our ON DELETE SET NULL parameter on the foreign key will mean that **the employees with the deleted department_id will have the department_id set to NULL**.

So, we can run this command:

```
DELETE FROM department
WHERE department_id = 3;
1 row deleted.
```

Our employee table will now look like this. Employee ID 2, which used to have a department ID of 3, now has a department ID of null. This is because department ID 3 was deleted, so all related records are set to null.

employee_id	first_name	department_id	(other columns…)
1	John	2	…
2	Sarah	(null)	…
3	Adam	1	…
4	Debbie	1	…

Example 5 – On Delete Cascade

This example will show you how the "on delete cascade" parameter works.

Our employee table would look like this:

```
CREATE TABLE employee (
  employee_id NUMBER(10),
  first_name VARCHAR2(20),
  last_name VARCHAR2(20),
  salary NUMBER(10),
  hire_date DATE,
  department_id NUMBER(10),
  CONSTRAINT fk_emp_deptid
  FOREIGN KEY (department_id)
  REFERENCES department(dept_id)
  ON DELETE CASCADE
);
```

We can see the **ON DELETE CASCADE** is written at the bottom.

Let's assume we have the same initial data setup as the earlier example.

Department

dept_id	department_name
1	Sales
2	HR
3	Finance

Employee

employee_id	first_name	department_id	(other columns…)
1	John	2	…
2	Sarah	3	…
3	Adam	1	…
4	Debbie	1	…

Now, let's delete department 3 again, and see what happens.

```
DELETE FROM department
WHERE department_id = 3;
1 row deleted.
```

Our employee table will now look like this. Employee ID 2, which used to have a department ID of 3, is now removed form the table entirely.

employee_id	first_name	department_id	(other columns…)
1	John	2	…
3	Adam	1	…
4	Debbie	1	…

This is what happens when we set ON DELETE CASCADE.

Example 6 – Alter Table

We can also add a foreign key constraint to a table by using the ALTER TABLE command.

This is good to use when a table is already created. We don't need to drop and recreate the table to get a foreign key constraint. We can just use ALTER TABLE and add it.

Let's assume we have the employee table created, but with no constraint:

```
CREATE TABLE employee (
  employee_id NUMBER(10),
  first_name VARCHAR2(200),
  last_name VARCHAR2(200),
  salary NUMBER(10),
  hire_date DATE,
  department_id NUMBER(10)
);
```

We can alter this table to add a constraint:

```
ALTER TABLE employee
ADD CONSTRAINT fk_emp_deptid
FOREIGN KEY (department_id)
REFERENCES department (dept_id);
```

This will add the constraint to the table.

What About Existing Data?

What if the table already has data in it?

Well, this is where we have the VALIDATE and NOVALIDATE keyword

Example 7 – Inserting Data

Our final example here for foreign key constraints is on inserting data.

We'll insert some data into the table we created earlier to see what happens.

Let’s assume we have the same initial data setup as the earlier example.

Department

dept_id	department_name
1	Sales
2	HR
3	Finance

Employee

employee_id	first_name	department_id	(other columns…)
1	John	2	…
2	Sarah	3	…
3	Adam	1	…
4	Debbie	1	…

Let’s insert a new employee:

```
INSERT INTO employee (employee_id, first_name, department_id)
VALUES (5, 'Mary', 2);
1 row(s) inserted.
```

This means Mary is inserted with department_id 2, which is HR.

What if we insert a value that doesn’t match a department?

```
INSERT INTO employee (employee_id, first_name, department_id)
VALUES (6, 'Kevin', 4);
```

We’ll get an error. This error means there is no matching record in the department_id table with an ID of 4. This is the exact kind of issue that the foreign key constraint prevents – bad data.

The user will need to either adjust the department_id mentioned (perhaps they meant department 3) or add a new department record with an id of 4 first.

3. Unique Constraint

A unique constraint is a type of constraint in SQL databases.

It defines a field or set of fields where the combination must be unique in a table.

So, if you create a unique constraint on one column, all of the values in that column must be unique. If you create a unique constraint on multiple columns, then the combination of those columns must be unique.

It's similar to a primary key constraint, except:

- A unique constraint can contain NULL values, but a primary key cannot
- A table can have more than one unique constraint, but only one primary key

Creating a Unique Constraint

How can you create a unique constraint?

```
CONSTRAINT constraint_name UNIQUE (columns)
```

This is the same whether it's defined inline or out of line.

Unique Constraint Restrictions

There are a few restrictions when creating a unique constraint:

- The columns in the unique constraint cannot be any of the following types: LOB, TIMESTAMP WITH TIME ZONE, LONG, LONG RAW, VARRAY, NESTED TABLE, REF, or a user-defined type.
- A composite unique constraint can't have more than 32 columns.
- You can't use the same columns for a unique constraint as a primary key.

Example 1 – Inline Unique Constraint

In this example, the government_id column is added to capture a government-issued ID number (such as a Social Security number in the US or a Tax File Number in Australia).

Creating a unique constraint inline is as simple as putting the word UNIQUE after the column definition.

```
CREATE TABLE employee (
  employee_id NUMBER(10),
  first_name VARCHAR2(20),
  last_name VARCHAR2(20),
  government_id VARCHAR(20) UNIQUE,
  salary NUMBER(10),
  hire_date DATE,
  department_id NUMBER(10)
);
```

This adds a unique constraint to the column. A name for the constraint is automatically generated by the database.

Example 2 – Out of Line Unique Constraint

This example shows you how to declare a unique constraint out of line:

```
CREATE TABLE employee (
  employee_id NUMBER(10),
  first_name VARCHAR2(20),
  last_name VARCHAR2(20),
  government_id VARCHAR(20),
  salary NUMBER(10),
  hire_date DATE,
  department_id NUMBER(10),
  CONSTRAINT uc_emp_govtid UNIQUE (government_id)
);
```

I've given it the name uc_emp_govtid as it follows my recommended pattern of [constraint type]_[table name]_[column name]. This makes it easy to see what object it is and what it refers to just by looking at its name.

Also, declaring a constraint out of line allows you to use multiple columns, which I'll show an example of shortly.

Example 3 – Unique Constraint with Multiple Columns

This example uses an out of line unique constraint on multiple columns. It enforces a rule that says "an employee's first name, last name, and hire date must be unique".

```
CREATE TABLE employee (
  employee_id NUMBER(10),
  first_name VARCHAR2(200),
  last_name VARCHAR2(200),
  government_id VARCHAR(20),
  salary NUMBER(10),
  hire_date DATE,
  department_id NUMBER(10),
  CONSTRAINT uc_emp_fnlnhd UNIQUE (first_name, last_name, hire_date)
);
```

The multiple columns are mentioned inside the brackets after the UNIQUE keyword, which ensures the combination of those three fields is met.

Example 4 – Alter Table

If you want to add a unique constraint to an existing table, you can do that with an ALTER TABLE command.

```
ALTER TABLE employee
ADD CONSTRAINT uc_emp_govtid UNIQUE (government_id);
```

This will add the unique constraint to the table after it's created. If there is data that already exists, then it needs to match this constraint rule (it needs to be unique). If it's not unique, an error is shown and the constraint is not created.

If you want to prevent the unique check from happening on existing data, you can use the VALIDATE or NOVALIDATE commands .

Example 5 – Inserting Data

Let's assume we have the employee table set up with our unique constraint on the government_id column, and let's insert some data.

```
CREATE TABLE employee (
  employee_id NUMBER(10),
  first_name VARCHAR2(200),
  last_name VARCHAR2(200),
  government_id VARCHAR(20),
  salary NUMBER(10),
  hire_date DATE,
  department_id NUMBER(10),
  CONSTRAINT uc_emp_govtid UNIQUE (government_id)
);

INSERT INTO employee
(employee_id, first_name, last_name, government_id, salary,
hire_date, department_id)
VALUES (1, 'John', 'Smith', '1002300', 55000, TO_DATE('23-JAN-2018',
'DD-MON-YYYY'), 1);
1 row(s) inserted.
```

This row is inserted successfully. Notice the government_id of 1002300.

The next row has the same government_id.

```
INSERT INTO employee
(employee_id, first_name, last_name, government_id, salary,
hire_date, department_id)
VALUES (2, 'Sarah', 'Jones', '1002300', 64000, TO_DATE('04-FEB-
2018', 'DD-MON-YYYY'), 3);
ORA-00001: unique constraint (ben.uc_emp_govtid) violated
```

This error has occurred because there is already a record that meets the unique constraint – a record that has the same government_id value. So this record is not inserted.

The ORA-00001 is an Oracle error, and we'll get a similar "unique constraint violated" error in other databases.

Let's try to insert another record with a different government_id.

```
INSERT INTO employee
(employee_id, first_name, last_name, government_id, salary,
hire_date, department_id)
VALUES (3, 'Mary', 'Stephenson', '1004091', 71500, TO_DATE('05-FEB-
2018', 'DD-MON-YYYY'), 2);
1 row(s) inserted.
```

This row is inserted successfully because the unique constraint is not violated (the government_id value is unique.

4. Not Null Constraint

A NOT NULL constraint is a type of constraint that means the specified column must have a value (it cannot be NULL).

It's a simple constraint and one that I see implemented quite a lot.

It improves the quality of your data, just like the other constraints do, but in a different way.

- **Creating a Not Null Constraint**

To create a NOT NULL constraint, simply add the words NOT NULL to the end of the column definition.

column_name data_type NOT NULL

A NOT NULL constraint must be declared inline. You can't declare it out of line.

- **Not Null Constraint Restrictions**

There are a few restrictions on NOT NULL constraints:

- You cannot declare a NOT NULL constraint out of line. It must be declared inline.
- NOT NULL constraints are also the only constraints you can specify inline on XMLType and VARRAY columns.

Example 1 – Inline Constraint

To create a NOT NULL constraint on a table, use the inline method.

```
CREATE TABLE employee (
  employee_id NUMBER(10),
  first_name VARCHAR2(20) NOT NULL,
  last_name VARCHAR2(20),
  government_id VARCHAR(20),
```

```
  salary NUMBER(10),
  hire_date DATE,
  department_id NUMBER(10)
);
```

This will ensure the first_name field cannot be null.

Example 2 – Out of Line

Let's try to create a NOT NULL constraint out of line.

```
CREATE TABLE employee (
  employee_id NUMBER(10),
  first_name VARCHAR2(200),
  last_name VARCHAR2(200),
  government_id VARCHAR(20),
  salary NUMBER(10),
  hire_date DATE,
  department_id NUMBER(10),
  CONSTRAINT nn_emp_fname NOT NULL (first_name)
);
ORA-00904:  invalid identifier
```

This shows an error as NOT NULL constraints cannot be created out of line. This is an Oracle error, and we'll get a similar error in other databases.

Example 3 – Alter Table

You can add a NOT NULL constraint to an existing column in a table by using the ALTER TABLE command. However, it's done using the MODIFY COLUMN rather than ADD CONSTRAINT.

```
ALTER TABLE employee
MODIFY (last_name CONSTRAINT nn_emp_lname NOT NULL);
```

This ensures the last_name cannot be null.

Example 4 – Inserting Data

Let's try inserting some data where a NOT NULL constraint is applied on the first_name column.

```
INSERT INTO employee
(employee_id, first_name, last_name, government_id, salary,
hire_date, department_id)
VALUES (1, NULL, Smith, '1002300', 55000, TO_DATE('23-JAN-2018',
'DD-MON-YYYY'), 1);
ORA-01400: cannot insert NULL into ("BEN"."EMPLOYEE"."FIRST_NAME")
```

As you can see, I've manually entered a NULL value here. This is an Oracle error, and we'll get a similar error in other databases.

We will also get a similar error if we omit the first_name column from the INSERT statement.

```
INSERT INTO employee
(employee_id, last_name, government_id, salary, hire_date,
department_id)
VALUES (1, Smith, '1002300', 55000, TO_DATE('23-JAN-2018', 'DD-MON-
YYYY'), 1);

ORA-01400: cannot insert NULL into("BEN"."EMPLOYEE"."FIRST_NAME")
```

This error happens because we didn't specify a value for this column, and a value is required because of the NOT NULL constraint.

5. Check Constraint

A check constraint is a type of constraint that ensures a column (or several columns) meets a specific condition.

Each row in the table must ensure the condition is true, or is unknown due to a NULL value.

Creating a Check Constraint

A check constraint can be created inline or out of line, and the syntax is the same:

```
CONSTRAINT constraint_name CHECK (conditions)
```

Let's look at what restrictions apply for a check constraint, and then look at some examples.

Check Constraint Restrictions

There are some restrictions when creating check constraints:

- You can't create a check constraint on a view. However, if you create a view using WITH CHECK OPTION, it performs a check in a similar way.
- You can only refer to columns in the same table, not other tables.
- Constraints cannot refer to:
    - Subqueries
    - Functions that are non-deterministic (which means they can get a different value each time they are called) such as CURRENT_DATE, SYSDATE, and other date-related functions.
    - User-defined functions
    - Pseudo columns such as CURRVAL, NEXTVAL, LEVEL, and ROWNUM
    - Dereferencing of REF columns
    - Nested table columns or attributes

Let's take a look at some examples.

Example 1 – Inline Check Constraint

This example shows how to create a check constraint using the inline method.

```
CREATE TABLE employee (
  employee_id NUMBER(10),
  first_name VARCHAR2(20),
  last_name VARCHAR2(20),
  government_id VARCHAR(20),
  salary NUMBER(10) CONSTRAINT ck_emp_salary
  CHECK (salary BETWEEN 10000 AND 500000),
  hire_date DATE,
  department_id NUMBER(10)
);
```

This constraint, called ck_emp_salary, ensures that the salary field is between 10,000 and 500,000. Any value that is attempted to be inserted outside of that range will fail the check constraint and won't be inserted into the table.

Example 2 – Out of Line Check Constraint

Declaring a check constraint out of line is done in a similar way.

```
CREATE TABLE employee (
  employee_id NUMBER(10),
  first_name VARCHAR2(20),
  last_name VARCHAR2(20),
  government_id VARCHAR(20),
  salary NUMBER(10),
  hire_date DATE,
  department_id NUMBER(10),
  CONSTRAINT ck_emp_salary
  CHECK (salary BETWEEN 10000 AND 500000)
);
```

Note that the constraint appears at the end of the column definitions.

Example 3 – Multiple Columns

A check constraint can also use multiple columns.

For example, this constraint ensures that the first name and last name are more than 10 characters combined.

```
CREATE TABLE employee (
  employee_id NUMBER(10),
```

```
  first_name VARCHAR2(20),
  last_name VARCHAR2(20),
  government_id VARCHAR(20),
  salary NUMBER(10),
  hire_date DATE,
  department_id NUMBER(10),
  CONSTRAINT ck_emp_name
  CHECK (LENGTH(first_name || last_name) > 10)
);
```

Example 4 – Alter Table

Just like with the other constraints, you can add a check constraint to an existing table using ALTER TABLE.

```
ALTER TABLE employee
ADD CONSTRAINT ck_emp_govtid
CHECK (LENGTH(government_id) > 3);
```

This check constraint ensures the government_id field is more than 3 characters long.

Example 5 – Inserting Data

Let's try to insert some data where a check constraint exists.

We'll use our first example for this.

```
CREATE TABLE employee (
  employee_id NUMBER(10),
  first_name VARCHAR2(20),
  last_name VARCHAR2(20),
  government_id VARCHAR(20),
  salary NUMBER(10) CONSTRAINT ck_emp_salary
  CHECK (salary BETWEEN 10000 AND 500000),
  hire_date DATE,
  department_id NUMBER(10)
);
Table created.
INSERT INTO employee
(employee_id, first_name, last_name, government_id, salary,
hire_date, department_id)
VALUES (1, 'John', Smith, '1002300', 55000, TO_DATE('23-JAN-2018',
'DD-MON-YYYY'), 1);
1 row(s) inserted.
INSERT INTO employee
(employee_id, first_name, last_name, government_id, salary,
hire_date, department_id)
VALUES (1, 'John', Smith, '1002300', 55000, TO_DATE('23-JAN-2018',
'DD-MON-YYYY'), 1);
```

The first record is inserted successfully.

```
INSERT INTO employee
(employee_id, first_name, last_name, government_id, salary,
hire_date, department_id)
VALUES (2, 'Sarah', 'Jones', '1002300', 6000, TO_DATE('04-FEB-2018',
'DD-MON-YYYY'), 3);
ORA-02290: check constraint ck_emp_salary violated
```

The second record shows an error because the value of 6000 for salary is outside the accepted range.

```
INSERT INTO employee
(employee_id, first_name, last_name, government_id, salary,
hire_date, department_id)
VALUES (3, 'Mary', 'Stephenson', '1004091', 705000, TO_DATE('05-FEB-
2018', 'DD-MON-YYYY'), 2);

ORA-02290: check constraint ck_emp_salary violated
```

This also shows an error, because the salary of 705,000 is also outside the range specified by the check constraint. This is an Oracle error, and we'll get a similar error in other databases.